SIRUS

LANDING ON PLANET

DiSHACLed

FOR MISSION

Validation report: toolchain specification



Sirus NV
Verlorenbroodstraat 122
9820 Merelbeke-Melle
info@sirus.be
+32 9 335 76 43
www.sirus.be

Thomas Carsten
30 September 2026
thomas.carsten@sirus.be

CONTENTS

Introduction 4

Motivation 5

Background and terminology 7

Who this report is for 7

Technologies 7

Terms used in this report 8

Part I: The architecture 9

1. Semantic model 10

1.1. How it works 10

1.2. What it supports 11

1.3. What it does not support 12

1.4. Future directions 12

2. Compilation runner 13

2.1. How it works 13

2.2. What it supports 14

2.3. What it does not support 14

2.4. Future directions 15

3. Validation principle 15

3.1. How it works 15

3.2. Data compatibility in theory and practice 15

3.3. What it supports 16

3.4. What it does not support 16

3.5. Future directions 17

4. Adding frameworks 17

4.1. Describing the framework's components 17

4.2. Writing the compilers 18

4.3. Writing the shapes 18

4.4. Testing the new framework 18

Part II: The reference implementation 19

Supported frameworks 19

Reference cases 19

5. Pipeline generator 20

5.1. How it works 20
5.1.1. Inputs and outputs *20*
5.1.2. Core compilers *20*
5.1.3. Framework compilers *20*
5.1.4. Example: compiling a pipeline *20*
5.2. What it supports 22
5.3. What it does not support 23
5.3.1. Not yet implemented *23*
5.3.2. Design choices of the reference implementation *24*
5.3.3. Limits set by the target frameworks *24*
5.4. Future directions 24
6. Pipeline validator 24
6.1. How it works 24
6.2. What it supports 25
6.3. What it does not support 25
6.4. Future directions 25
Part III: Assessment 26
7. Status at a glance 26
8. Conclusion 27
References 30
Appendix A. Complete list of checks 31

INTRODUCTION

This report describes what the toolchain specification delivers today and where its limits lie. The toolchain consists of two layers, which the report treats separately. The **architecture**, described in Part I, is the general and framework-independent foundation. It comprises a semantic model for describing pipelines, a *compilation runner* that turns such descriptions into runnable builds, and a principle for validating descriptions before they are trusted. The **reference implementation**, described in Part II, is one concrete use of that architecture: a pipeline generator and a pipeline validator for four frameworks, namely RDF-Connect, LDIO, Apache NiFi and semantic.works. It serves as a proof of concept, and it is the means by which the architecture has been tested in practice.

The separation matters for reading the limitations. A limitation of the reference implementation can be lifted by writing or changing compilers, without touching the architecture, whereas a limitation of the architecture constrains every possible implementation. Throughout, the report focuses on functionality and limitations as experienced by users of the toolchain rather than on implementation details.

MOTIVATION

Linked Data tooling is largely community-driven and diverse. Frameworks such as LDIO, RDF-Connect, Apache NiFi and semantic.works each work well within their own boundaries, but building a pipeline in practice is still mostly manual work. Developers have to find suitable components across many repositories, read their documentation, install them, write configuration in each framework's own format, and discover the often implicit assumptions components make about each other. Every time a pipeline is moved, replicated or extended with a component from another framework, much of this work starts again.

The toolchain specification was built to change this. Its central idea is to describe a pipeline **once**, as a framework-independent semantic description, and to derive everything else from that description by machine. Seven goals follow from this idea.

The first goal is **interoperability**. A single pipeline should be able to combine components from different frameworks, with the tool working out how data passes from one framework to the next, so that users can choose components for what they do rather than for the framework they happen to belong to.

The second goal is **replicability**. A pipeline description should be enough to rebuild the pipeline anywhere and at any time, without relying on knowledge that exists only in the head of whoever built it first. This makes it easy to regenerate a pipeline after a change, to share it with others, and to reuse it as the starting point for a new one.

The third goal is **automatic validation**. As far as possible, errors should be caught before a pipeline is deployed instead of being discovered by trial and error at run time. To this end, components should state their requirements and constraints explicitly, so that a proposed pipeline can be checked against them automatically. This includes inconsistencies between frameworks, which no single framework can detect on its own.

The fourth goal is **automatic installation**. Everything a pipeline needs should be pulled in automatically from public sources: container images from public registries and software packages from public package repositories. Because every component runs in a Docker container, a generated pipeline should start with a single command on any machine that has Docker, without any manual installation.

The fifth goal is **ease of use**. As much work as possible should be taken off the pipeline author. Dependency resolution, the assignment of components to containers, the connections between containers, ports and framework-specific configuration formats should all be handled by the tool. The pipeline author describes *what* the pipeline should do, and the tool decides *how* it is built.

The sixth goal concerns **semantics**. Pipelines and their components should be first-class, machine-readable metadata rather than configuration files buried in a project folder. Described in RDF with established vocabularies, they become useful beyond execution: for provenance, because a dataset can be linked to the pipeline and the transformation steps that produced it; for discoverability, because components and pipelines can be published in catalogs, found and reused by a community; and for analysis, because pipelines can be queried, compared and reasoned over like any other Linked Data.

The seventh goal is **modularity**. The design should be easy to extend and to adapt to specific use cases. Components are reusable units in a shared catalog, and the knowledge about each framework is packaged in small, independent pieces, so that adding a component, a framework or a new check does not require changes to what already exists.

The report is organised as follows. The Background section introduces the technologies and terms used throughout. Part I describes the architecture, including how new frameworks are added to it, and Part II describes the reference implementation together with the reference cases against which it was tested. The main sections of both parts follow the same order: how it works, what it supports, what it does not support, and future directions. Part III summarises the result per goal and reflects on how much of the original ambition was achieved.

Every limitation in the report is assigned to one of three categories. A limitation is **not yet implemented** when the missing feature fits the current design and could simply be added. It is a **design choice of the reference implementation** when it follows from a deliberate decision in the reference compilers, which other compilers

could make differently without any change to the architecture. Finally, it is **architecture-inherent** when it follows from the architecture itself, or from something outside the toolchain on which the architecture builds, such as SHACL or the target frameworks. Only this last category applies to every possible implementation.

BACKGROUND AND TERMINOLOGY

This section introduces the technologies and terms on which the rest of the report relies. Readers familiar with Linked Data pipelines can skip it.

WHO THIS REPORT IS FOR

The toolchain involves three roles, and most of the report concerns the first. *Pipeline authors* describe a pipeline and generate it; they need to know what can be described (§1), what the generator produces (§5) and what the validator checks (§6). *Catalog maintainers* add components to the catalog so that pipeline authors can use them. *Framework integrators* are the compiler developers who add support for a new framework, whether by extending the reference implementation or with a set of compilers of their own. Part I, and §4 in particular, is written for them. Figure 1 shows how the three roles come together.

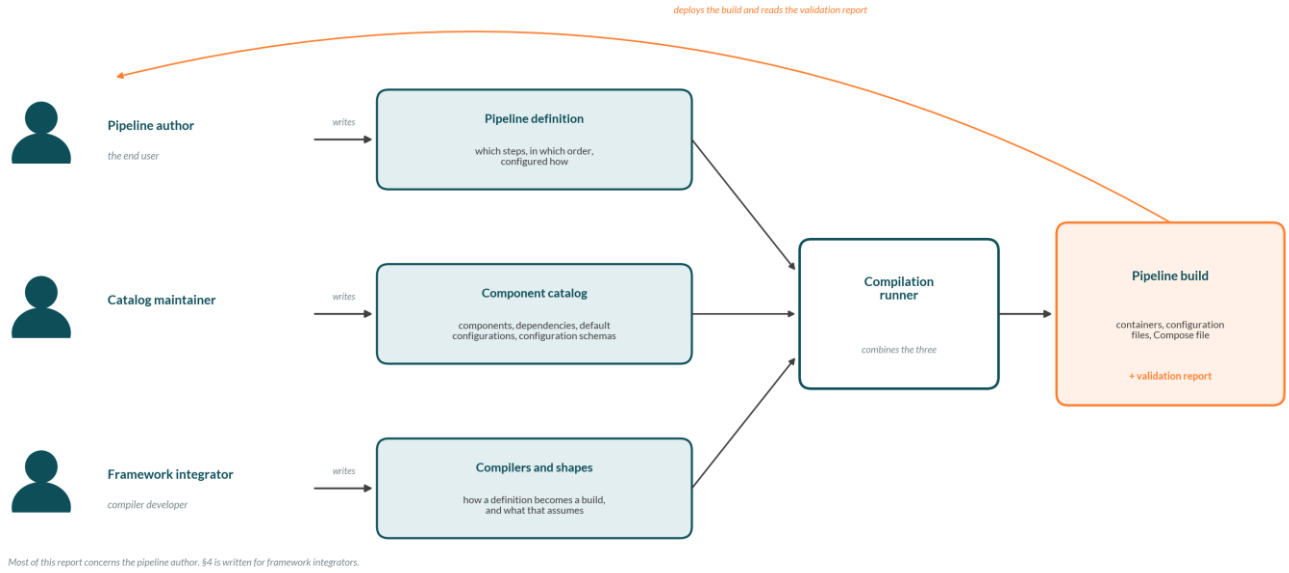


Figure 1. The three roles in the toolchain. The pipeline author writes a pipeline definition, the catalog maintainer describes components in the catalog, and the framework integrator writes the compilers and shapes that turn a definition into a build. The compilation runner combines the three, and the pipeline author deploys the result and reads its validation report.

TECHNOLOGIES

A **pipeline** is a chain of processing steps through which data flows. A typical pipeline fetches data from a web API, converts it to another format, checks it and stores it in a database.

The toolchain processes **Linked Data**, which is data published on the web in **RDF** [1]. RDF describes data as statements of the form *subject – property – value*. The toolchain also uses RDF to describe the pipelines themselves, and the examples in this report are written in **Turtle** [2], a text notation for RDF. RDF data is stored in a **triple store**, a database queried with the language **SPARQL** [3]; a *SPARQL update* is a request that writes data into such a store. **LDES** [4] (Linked Data Event Streams), which appears in several reference cases, is a standard for publishing a stream of changing Linked Data on the web.

SHACL [5] is a W3C standard for writing rules about what RDF data must look like, such as "every step has exactly one component". A single rule is called a **shape**, and software can check data against shapes automatically.

Docker runs software in isolated *containers*, each started from a ready-made *image*. **Docker Compose** [6] starts several containers together from a single file, `docker-compose.yml`. Because every component runs in a container, a generated pipeline requires no installation besides Docker itself. The software inside the containers is installed from public **package repositories**, such as npm for JavaScript, PyPI for Python and Maven for Java, while the container images themselves come from similar public registries.

TERMS USED IN THIS REPORT

Term	Meaning
Framework	Existing software that runs pipelines, such as LDIO or Apache NiFi. The toolchain does not replace frameworks; it generates their configuration, the environment they need in order to run and, at least in part, the wiring between them. The frameworks of the reference implementation are described in Part II.
Orchestrator	A framework's main program, which runs its pipelines, usually inside one container.
Component	A reusable building block, such as "poll a URL" in LDIO or "log to console" in RDF-Connect.
Catalog	The collection of all components the toolchain knows, with everything needed to use them.
Pipeline definition	The description of one pipeline, written by the pipeline author. It may contain several **plans**: separate pipelines described together, for example a publisher of data and a consumer of that data.
Step	One use of a component in a pipeline, with its own configuration. The same component can be used in several steps.
Channel	A connection between steps along which data flows.
Step configuration	The settings a pipeline author gives a step, for example which URL to poll.
Default configuration	A configuration file stored with a component in the catalog, used for every pipeline that includes the component. A *Compose description* is one such file: the part of a docker-compose.yml that states how the component's container is started.
Build	Everything produced for one pipeline: containers, configuration files and a record of how they were made.
Compiler	A small piece of software responsible for one task in producing a build, such as writing LDIO's configuration file. It reads the build description, adds its own contribution to it, and declares the condition under which it runs (§2.1).
Compilation runner	The part of the architecture that decides when each compiler runs (§2).
Bridge	The pair of components that carries data from one container to another: a *sender* in the first container and a *receiver* in the second. Components that can play these roles are called **boundary components**.
Segment	A stretch of connected steps inside one container, through which data flows without leaving the container.
Multi-tenant	Describes a framework that can run several independent pipelines side by side in one container. LDIO and RDF-Connect are multi-tenant; semantic.works is not.

PART I: THE ARCHITECTURE

From the pipeline author's point of view, working with the toolchain takes three steps. First, the pipeline author describes the pipeline semantically: which components take part, in which order data flows between them, and how each component is configured. The components are chosen from a catalog, and the pipeline author does not need to know how they are installed, which containers they run in or how they reach each other. Second, the pipeline author hands this description to the toolchain, which compiles it into a complete Docker project: a set of containers wired into one network, with every configuration file each framework needs and a Compose file that starts them all. Third, the project comes with a validation report, which tells the pipeline author before deployment whether the pipeline violates any known constraint and, if so, where. The same check can also be run on its own, before anything is generated. If the report is clean, a single `docker compose up` starts the pipeline.

The architecture makes this possible through three parts, each responsible for one stage of this workflow. The **semantic model** (§1) provides the vocabulary for the first step. It lets the pipeline author describe a pipeline in framework-independent terms, and lets catalog maintainers describe once, for every component, everything needed to deploy it. The **compilation runner** (§2) accomplishes the second step. It coordinates many small, independent compilers, each of which contributes one part of the build, such as resolving dependencies, assigning components to containers, connecting containers or writing a framework's configuration file, until the build is complete. The **validation principle** (§3) underlies the third step. Constraints are attached to components as machine-checkable rules and checked against the build as it will actually be produced. §4 then describes how support for a new framework is added on this foundation.

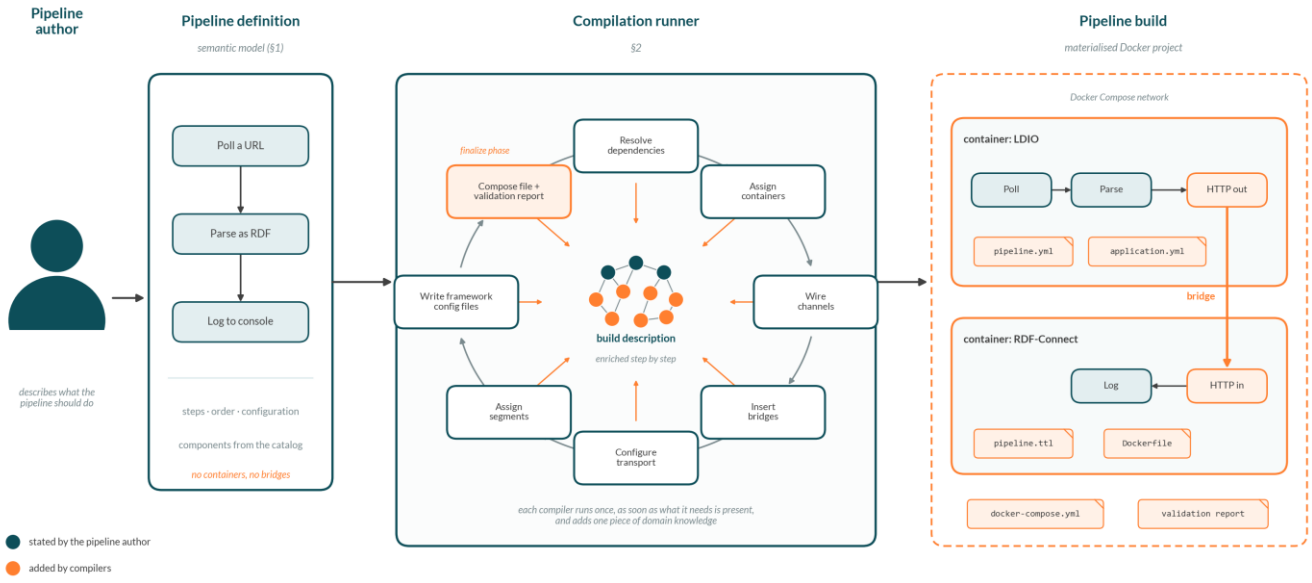


Figure 2. From intent to deployable build. The pipeline author states only the steps, their order and their configuration (dark). The compilation runner hands the growing build description to one compiler after another, each adding one piece of domain knowledge (orange), until the build is materialised as a Docker project: containers, the configuration files of each framework, the bridge between them, a Compose file and a validation report. The frameworks shown are those of the running example in §1.1.

None of these parts contains knowledge of a particular framework, of how components are assigned to containers, or of how configuration files are laid out. All such knowledge lives in compilers, catalog entries and shapes, which belong to an implementation. This is what makes the architecture open to change. A developer can add compilers to support a new framework, or modify existing compilers to suit a specific use case. The underlying principles remain the same in every case. A compiler always translates the intent expressed in the semantic model into a concrete implementation, and in doing so it captures a piece of domain knowledge that would otherwise have to be applied by hand. The reference implementation described in Part II is one such set of compilers.

1. SEMANTIC MODEL

1.1. HOW IT WORKS

The semantic model, the *toolchain* ontology with the prefix `tcs:` [7], is the vocabulary in which pipelines, components and builds are described. Wherever possible it reuses established vocabularies, each for a clearly delimited purpose. **P-Plan** [8] describes the plan of a pipeline and its steps, that is, the pipeline author's intent. **DCAT** [9] describes the catalog and the pipeline components in it as reusable resources. **PROV-O** [10] records provenance, linking a build to the plan it implements and, through it, a dataset to the pipeline that produced it. **SPDX** [11] describes software packages and their versions, which is what makes automatic dependency resolution possible, and it also describes the files a build consists of. **SHACL**, finally, expresses the expectations and constraints that come with each pipeline component: the configuration it expects, the data it expects to receive and produce, or the position it may take in a pipeline. The model can be used on its own as a common language for describing pipelines, even without any software that acts on the descriptions.

The model separates four concerns. The **component catalog** holds everything known about the available components: what each component depends on (other components and software packages), where it is documented, which ready-made configurations it comes with (defaults, Docker Compose descriptions, Dockerfiles), and which constraints apply when it is used. This information is written once, in the catalog, and never has to be repeated in a pipeline. The **pipeline definition** is a *plan* for a pipeline. It consists of *steps*, each of which points to the catalog component it uses and carries that step's configuration. Steps are connected with each other to indicate the order in which data passes through them and where the pipeline branches. The **pipeline generator** is the software that turns a definition into a build with the help of the catalog; the model deliberately leaves open how it does so. The **pipeline build**, finally, is the result: which containers run, which steps each container executes, and which files were generated. It is described in the same vocabulary as the definition.

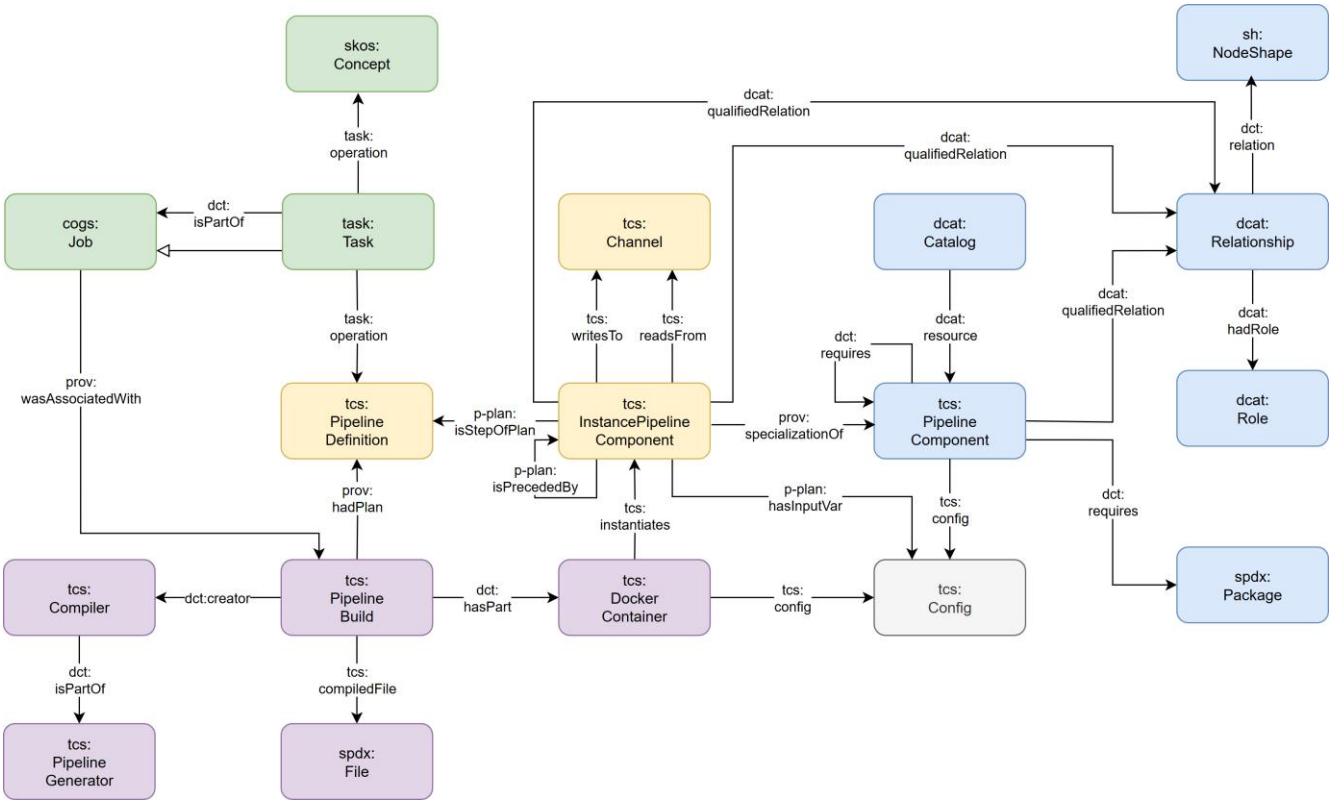


Figure 3. The semantic model. Yellow: the pipeline definition, a P-Plan plan whose steps (tcs:InstancePipelineComponent) are connected by channels and each specialise a catalog component. Blue: the component catalog, recording for each component its dependencies, its configurations and its constraints, the latter as SHACL shapes, each attached with a statement of what it constrains. Purple: the pipeline build, with its containers, the files generated for it and the compilers that produced it. Grey: configurations, used by steps, components and containers alike. Green: the task a pipeline carries out, which links a definition to the kind of operation it performs and a job to the build that performs it.

This separation keeps pipeline definitions small. Consider a pipeline that fetches a public RDF document every five seconds with LDIO, parses it, and logs each result with RDF-Connect. Its complete definition, written in Turtle with prefixes omitted, is shown below. The components it uses come from the catalog of the reference implementation. The first block declares the pipeline, and each further block describes one step: `prov:specializationOf` names the catalog component the step uses, `p-plan:isStepOfPlan` names the pipeline the step belongs to, `p-plan:hasInputVar` holds the step's configuration, and `p-plan:isPrecededBy` names the previous step.

```
demo:SemanticsDemoPipeline a tcs:PipelineDefinition .

# Poll: fetch a public RDF document every five seconds (LDIO).
demo:Poll a tcs:InstancePipelineComponent ;
    prov:specializationOf ldio:HttpInPoller ;
    p-plan:isStepOfPlan demo:SemanticsDemoPipeline ;
    p-plan:hasInputVar [ a tcs:PipelineConfig ;
        tcs:embedded [ ldio:url  "https://www.w3.org/People/Berners-Lee/card.ttl" ;
                       ldio:cron "*/5 * * * * *" ] ] .

# Parse: turn the fetched document into RDF (LDIO).
demo:Parse a tcs:InstancePipelineComponent ;
    prov:specializationOf ldio:RdfAdapter ;
    p-plan:isStepOfPlan demo:SemanticsDemoPipeline ;
    p-plan:isPrecededBy demo:Poll .

# Log: print each result to the console (RDF-Connect).
demo:Log a tcs:InstancePipelineComponent ;
    prov:specializationOf rdfc:LogProcessorJs ;
    p-plan:isStepOfPlan demo:SemanticsDemoPipeline ;
    p-plan:isPrecededBy demo:Parse .
```

The definition contains no containers, no ports and no Docker images, and it says nothing about how data gets from LDIO to RDF-Connect. Deriving all of this is the task of an implementation; §5.1.4 shows how the reference implementation does so for the larger demonstrator pipeline.

Constraints are part of the description. SHACL shapes can be attached to components, steps or channels, and each attachment states what the shape constrains: the configuration a component expects, or the data it expects to receive or produce. The validation principle (§3) builds on these shapes.

Framework-specific vocabulary stays outside the model. The `tcs:` vocabulary is shared by all frameworks, while everything framework-specific, such as a component's role in LDIO, the location of an RDF-Connect processor's definition, or the names of configuration parameters, is expressed in the framework's own vocabulary within catalog entries and step configurations.

1.2. WHAT IT SUPPORTS

The model gives pipelines a framework-independent structure: steps from any framework are described in the same way and can be combined in one pipeline. Data flow can be linear, written tersely with "is preceded by", or branching, in which case explicit channels describe which step feeds which. One description can hold several related plans, such as a publisher and its consumer.

The model can describe any pipeline whose steps form a directed acyclic graph (DAG) [12]: data flows along directed connections from step to step and never returns to a step it has already passed. Each step can have any number of inputs and outputs, so pipelines can branch and fan out. Three conditions apply: all steps must be connected to each other directly or indirectly, data must never flow in a circle, and every step must be based on a component in the catalog. Whether a described pipeline can also be *run* depends in addition on the implementation and on the target frameworks (§5.3).

Components are reusable and declare their dependencies down to individual software packages and versions. Configurations can take any shape: they can be written directly in RDF as nested key/value data, or embedded as a text body in a stated format such as YAML, JSON or Turtle. Configurations are classified by what they concern (defaults, container start-up, image build, pipeline behaviour), so that each compiler can select the ones relevant to it. Constraints can be attached to components, steps and channels, as described in §1.1.

The model also covers the transport between containers. Components can be marked as receiving or sending endpoints for such traffic, together with the transport they speak, currently HTTP and SPARQL update over HTTP. Sensitive values such as passwords are referenced by the name of an environment variable and never stated in the description itself.

Finally, builds are described in the same vocabulary as definitions. A build records its containers, which components and steps each container runs, every generated file with its path and contents, which pipeline definition it implements, and which compilers produced it. A dataset can thus be traced back to the pipeline that produced it.

1.3. WHAT IT DOES NOT SUPPORT

Most gaps in the model are **not yet implemented**: the vocabulary could be extended to cover them. A pipeline cannot yet replace a catalog default with its own file. Transports other than HTTP, such as message brokers like Kafka, are anticipated by the model's design but have no vocabulary yet. Event-driven subscriptions, such as "notify this service whenever an error record appears in the triple store", are not point-to-point data flows and fall outside the current channel model. The channel concept leaves room for conditional routing ("on success", "on failure"), but no shared vocabulary for it exists, and NiFi relationships are therefore expressed in NiFi's own terms.

Configuration is expressed in each framework's own terms, and this is by design: every framework carries full responsibility for the configuration it expects, so a step's configuration uses the properties of its component's framework, such as LDIO's `ldio:url` and `ldio:cron` in the example above. What the model does not yet offer is support for the pipeline author in writing this configuration. It does not distinguish the configuration an author is expected to provide from the configuration a compiler needs or adds itself (§3.4), and it offers no pre-configured templates that would give authors a sensible starting point for each component. In addition, when a component has several named output slots, the framework-neutral description cannot say which channel feeds which slot, so this has to be stated in the framework-specific configuration.

The model also lacks vocabulary for deployment. It cannot express how many instances of a component should run, which steps should be isolated from each other, or resource limits and scaling; the reference implementation derives container layout from the catalog alone (§5.3.2). It describes a build as a set of Docker containers and has no vocabulary for other targets, such as an existing server. The reference implementation's remote NiFi deployment works around this with a container whose only task is to upload the flow. Finally, there is no mechanism yet to relate descriptions written against different versions of the model.

One limitation is architecture-inherent. SHACL describes what data looks like, not how a component changes it, so the model cannot express the transformation a component performs on the data it receives. §3.2 explains why SHACL is not suited to this and what it means for validation.

1.4. FUTURE DIRECTIONS

The most pressing extension is an override mechanism that lets a pipeline replace a catalog default with its own configuration. Beyond that, the model would benefit from additional channel types for non-HTTP transports, a way to describe event-driven subscriptions across frameworks, and a framework-neutral vocabulary for conditional routing. Versioning of the model would allow descriptions written against different versions to be related to each other, complementing the version mapping discussed in §2.4. Alignment with the discovery specification would make pipelines and components described in the toolchain vocabulary discoverable across data spaces.

Finally, the model opens up new ways of authoring pipelines. Today, a pipeline definition is written by hand in RDF, which assumes a human author who knows the semantic model. Because the definition is a machine-readable interface, it can equally be produced by other means: by a graphical front-end in which the author draws the pipeline, or by a language model that writes the definition from a request in plain language. The latter fits the toolchain particularly well, because every definition is checked against the built-in shapes before it is built, so errors in a generated definition are caught automatically. Either extension would make the toolchain considerably easier to use.

2. COMPILATION RUNNER

2.1. HOW IT WORKS

Producing a build is not the job of a single large program. It is divided among many small compilers, which are coordinated by a compilation runner. This division is deliberate. It keeps the toolchain modular, because each compiler can be added, replaced or modified on its own, and it keeps it scalable in scope, because new functionality arrives as new compilers rather than as growing complexity in one program. A compiler is a small piece of software that takes the build description, an RDF graph, as its input, adds its own contribution to it, and declares the condition under which it is to run. The runner knows nothing about frameworks, containers or file formats; its only task is to decide when each compiler runs. Several principles govern how this works.

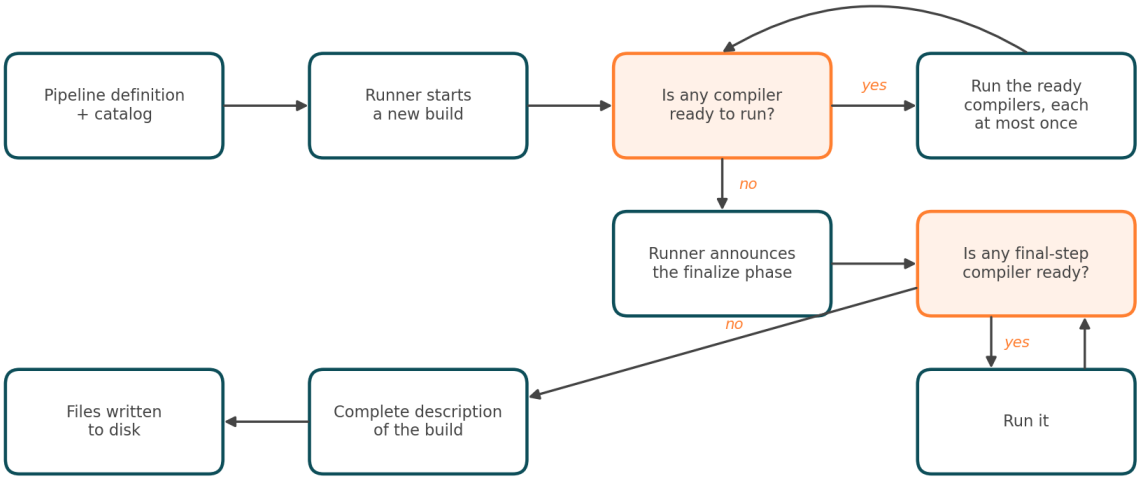
Each compiler has a single responsibility and embodies one piece of domain knowledge. In the reference implementation, for example, one compiler knows how to assign components to containers, another how an LDIO pipeline file is laid out, and yet another how to assemble the `docker-compose.yml`. Knowledge about a framework therefore lives in a small number of clearly named places instead of being spread throughout the tool.

Compilers decide for themselves when they are to run. Each compiler declares a condition on the current state of the build, such as "there is a container running the LDIO orchestrator" or "container assignment has finished". The runner never calls a compiler by name. Instead, it repeatedly asks every compiler whether its condition is met, runs the ones that answer yes, and continues until no compiler has anything left to do. The order of execution therefore emerges from the conditions: a compiler that needs the output of another simply waits until that output appears. Each compiler runs at most once per build. This is a deliberate choice: it keeps the order of execution traceable, and the contribution of each compiler becomes immediately clear when one inspects how it changed the build description.

Some work can only be done once everything else has settled. A file that lists every container can only be written when all containers are known, and validation should examine the finished build. For this purpose the runner signals phases. When no compiler is ready any more, it announces a *finalize phase*, at which point compilers waiting for this signal become ready. When they have run, the runner removes its own bookkeeping from the result.

Compilers never call each other. The build is an RDF graph shared by all compilers: one compiler writes a fact into it, and another reads it. The principle is intentionally kept simple. As a consequence, adding a compiler never requires changing the others or the runner. Files are part of this graph as well. Compilers do not write to disk; they attach each file they produce, with its name, location and contents, to the build. Only after the last compiler has run are the files written to the output folder.

Which compilers take part is decided by a **configuration**, which lists the compilers along with the files to read, such as the catalog, the pipeline definitions and the shapes. The same runner with different configurations yields different tools, and the reference implementation uses this to build its generator and its validator from largely the same compilers (§5, §6). The diagram below summarises the process.



2.2. WHAT IT SUPPORTS

Because the runner has no built-in knowledge of frameworks or output formats, it can coordinate compilers for any framework and any kind of file; the four frameworks of Part II are examples, not requirements. It can be extended without modification: a new compiler is added to a configuration and declares its own condition, and neither the runner nor any other compiler has to change. Different configurations produce different tools from one set of compilers, and because the pipeline to compile is passed to the runner for each run, a single configuration can be reused for every pipeline in its files.

The runner also makes builds transparent. Each build records which compilers ran and in which order. Because files are part of the build, the complete result and each compiler's intermediate state can be examined before anything is written to disk, and files can only be written inside the chosen output folder.

Extending the toolchain means writing compilers, and this is intended. Whatever an expert applies by hand today to turn a pipeline design into a working implementation is captured once in a compiler, after which it is applied automatically. Adding a framework is therefore developer work, but it never requires changing existing code.

2.3. WHAT IT DOES NOT SUPPORT

The runner compiles one plan per run. This is a deliberate choice rather than a limitation, because it keeps each compilation self-contained and traceable. A pipeline definition containing a publisher and a consumer, for example, is compiled by running the generator once for each plan, which produces two separate projects.

Three features are not yet implemented. First, compilers read the pipeline description directly in the vocabulary of the semantic model, so any change to the model affects every compiler that reads the changed terms. A translation layer between published versions of the model and the version the compilers expect has not been built yet. Second, the runner is not yet optimised for scale. The complete catalog is loaded into memory for every run, and in every round each compiler is asked again whether its condition is met. Both can be solved without changing the principles above, for example by querying components from the catalog only when they are needed and by asking again only those compilers whose inputs have changed. Third, every build is produced from scratch. A build cannot yet include services that are already running, such as a triple store that is already up; everything the pipeline needs is started anew.

Several properties are architecture-inherent. Since the order of execution is never stated explicitly but emerges from the compilers' conditions, compilers that become ready at the same moment must not interfere with each other, and a compiler whose condition is never met is simply not run, without any error being raised. Writing correct conditions is therefore the responsibility of whoever writes the compilers. Because each compiler runs only once, it cannot react to changes made after it ran; work that depends on such changes has to be placed in a separate compiler that waits for them. Every compiler also relies on certain properties being present in the description, so a build is only as correct as these assumptions are complete, and the assumptions are only

checked if they are written down as shapes (§3). There is currently no way to state which constraints a particular compiler assumes, so replacing or modifying a compiler risks a drift between the compilers and the shapes that are meant to check them (§3.4).

2.4. FUTURE DIRECTIONS

A version-mapping layer between published versions of the semantic model and the version compilers expect would allow the two to evolve independently. A build log, recording for every compiler when it became ready and what it added to the build description, would make the traceability of the runner directly available to pipeline authors. Loading components from the catalog on demand, and asking again only the compilers whose inputs have changed, would let the runner scale to large catalogs and pipelines. Allowing a build to include services that are already running would let generated pipelines reuse existing infrastructure.

One refinement of the phases has been prototyped on a development branch of the reference implementation. A compiler that runs late, such as a bridging compiler, can add components that need containers of their own, and files that list all requirements of a container, such as a Dockerfile, must only be written once these are known. The prototype therefore lets the set of required components settle completely before such files are written.

3. VALIDATION PRINCIPLE

3.1. HOW IT WORKS

Constraints are part of the description (§1.1): SHACL shapes attached to catalog components, steps or channels. Implementations add further shapes that express what their compilers expect. Validation checks a pipeline against all of these shapes before the build is trusted.

Validation itself is carried out by a compiler, the validation-report compiler. It waits for the finalize phase of the runner (§2.1), so that it examines the build as it will actually be produced, including components that other compilers added on their own. The compiler takes part in two tools. The pipeline generator runs it at the end of every generation, so that every generated project contains a validation report. The pipeline validator is a runner configuration of its own: the compilers that shape the build, followed by the validation-report compiler, but none of the compilers that write framework files.

The resulting validation report has two parts. The first part consists of constraint checks. Every shape that applies to the build is evaluated, and the report lists for each shape what it checks, in plain language, and whether it passed. Violations name the offending part of the description and explain the problem. The second part is reserved for data-compatibility checks, which compare the data one component produces with the data the next one expects. They are not yet fully implemented, and §3.2 explains why their practical use is limited.

A failed validation does not block the build. The report marks the result as invalid, and the pipeline author decides how to proceed. This is a deliberate choice: the pipeline author can still inspect the generated project and correct the problem directly in its files. It reflects a more general principle of the toolchain. The toolchain takes as many decisions as possible off the pipeline author, but it never has the last word: the generated output consists of ordinary, readable files, and the author can always adjust or overwrite them afterwards. This also lets the author work around current limitations, for example by pointing the generated Compose file at a service that is already running (§2.3).

3.2. DATA COMPATIBILITY IN THEORY AND PRACTICE

In theory, the semantic model allows a stronger kind of validation than constraint checks. If every component described, as SHACL shapes, the data it expects to receive and the data it produces, the validator could compare, for every connection, the sender's output description with the receiver's input description, and so detect before deployment that two components do not fit together. Such shapes are called *data descriptions* in this report, and comparing two of them is called *shape matching*. The reference implementation prepares this check: data descriptions are collected and paired for every connection, but the library that compares two shapes is still being developed, so no verdicts are produced yet.

In practice, this kind of check is rarely possible. Most components cannot state in advance which data they expect or produce, because their behaviour is not fixed: it is defined by their configuration. A SPARQL construct step produces whatever its query constructs, a mapping step produces whatever its mapping describes, and an HTTP poller returns whatever the polled source serves. What such a component receives and produces can only be known once it is configured, and often only once it runs.

SHACL cannot bridge this gap, and the reason lies in what SHACL is. A SHACL shape is a static description: it states what one graph must look like, and a validator checks one graph against it. SHACL has no notion of a function from input to output, and a shape cannot depend on the configuration of the component it describes. How a component changes data, for example that it renames a property, adds a timestamp or keeps only some records, is a transformation, and transformations are not something SHACL can express. Describing them would require a language for transformations, or shapes derived automatically from each component's configuration, which is a research question in its own right.

Shape matching is therefore of limited practical use, even once the comparison library is available. It works for components whose input or output is fixed, such as a component that always produces observations in one standard form, and for connections where the pipeline author states explicitly what data will flow. It also helps only if these descriptions are representative of the data that actually flows through the pipeline; a description that does not match reality makes the check worthless. The project investigated this route and concluded that it is closer to a theoretical exercise than to a practical tool. The value of validation lies mainly in the constraint checks.

3.3. WHAT IT SUPPORTS

The principle accepts any constraint SHACL can express, including checks that span several steps, such as "data never flows in a circle". Constraints are kept next to what they constrain: a component's configuration schema sits in its catalog entry, and an implementation's rules sit next to its compilers. Because validation runs on the build as it will be produced rather than on the description as written, automatically added components are checked as well. The reports are readable, since every shape carries a message stating what it checks, and the report itself is a file in the build.

3.4. WHAT IT DOES NOT SUPPORT

Five capabilities are not yet implemented. Comparing an output description with an input description requires a shape-matching library, which is still being developed; until it is available, the pairs are collected but cannot be judged (§3.2). The report does not yet state which steps or connections went unchecked because no shape covers them, so a pipeline author cannot tell from the report alone how complete the validation was. Validation is also static: it checks the description, not the data that later flows through the running pipeline.

Validation also does not yet distinguish between what the pipeline author has to configure and what a compiler needs in order to work. Both are expressed in the configuration schema of a component, so a single schema has to serve two audiences, and the report cannot tell the author whether a violation concerns their own input or an assumption of the tooling. A split into user-facing and compiler-facing configuration, with a translation from the one to the other, has been prototyped on a development branch of the reference implementation, but no catalog entry uses it yet.

Finally, a constraint cannot yet be tied to the compiler that introduces it. Many constraints exist only because a particular compiler relies on them. The rule that data on a connection may cross into at most one other container, for example, reflects the limits of one bridging compiler, not of the semantic model. At present such constraints are attached to components or selected by framework, and they apply whenever those components appear, regardless of which compilers take part in the run. If an integrator replaces a compiler with one that lifts a restriction, the old constraint keeps firing until it is removed by hand. The report also cannot tell the pipeline author which compiler a violated constraint belongs to. The semantic model already anticipates attaching shapes to compilers, but validation does not yet make use of it.

Two limitations are architecture-inherent. Only what has been written down can be checked, so a passing report means that no known constraint is violated, not that the pipeline will work. And because SHACL describes data rather than transformations, what a component does to the data, as determined by its configuration, is out of

reach of static validation (§3.2). Some errors in transformation logic, such as a wrong query or a wrong target graph, can therefore only be found when the pipeline runs.

These limits should be seen against the status quo. Only what is expressed in SHACL can be validated, but SHACL combined with the pipeline definition allows constraints to be stated explicitly, across frameworks, and checked automatically before deployment. In the frameworks themselves, validation is usually limited to what each framework checks when it loads its own configuration, and inconsistencies between frameworks are not checked at all.

3.5. FUTURE DIRECTIONS

Once the shape-matching library is available, the data-compatibility checks can deliver verdicts for the connections where both sides are described, although §3.2 explains why these will remain few. Reporting coverage, with warnings for steps and connections that could not be checked, would let pipeline authors judge how much a passing report means. Separating user-facing from compiler-facing configuration, as prototyped, would let the report tell the author which violations concern their own input. Attaching constraints to the compilers that introduce them would make validation follow the configuration: a constraint would then apply only when its compiler takes part in the run, and the report could name the compiler behind each violation. This matters most when compilers are replaced or modified for a specific use case (Part I introduction). A further step is run-time data validation: validation steps could be inserted automatically into a generated pipeline, for example at its start and end, so that data flowing through it is checked against the same shapes used at design time.

4. ADDING FRAMEWORKS

This section is written for framework integrators, the developers who add support for a new framework. Pipeline authors can skip it. It is included because it shows what extending the toolchain takes, which bears on the modularity goal.

In the architecture, a framework is nothing more than a set of catalog entries, compilers and shapes. Adding one therefore never requires changing the runner, the semantic model or the compilers of other frameworks. An integrator can start from nothing and write every compiler, including those that assign containers or bridge between them, or can reuse compilers that already exist. The frameworks of the reference implementation (Part II) were added in exactly this way, and its framework-independent core compilers (§5.1.2) are available for reuse. Whichever route is taken, adding a framework involves three activities: describing the framework's components, writing the compilers that produce its files, and writing the shapes that make the compilers' assumptions checkable. The examples below are taken from the reference implementation.

4.1. DESCRIBING THE FRAMEWORK'S COMPONENTS

The first activity is to describe the framework's components in the catalog using the semantic model: what each component is, what it depends on, where it is documented, and which ready-made configurations it comes with, such as a Docker Compose description, a Dockerfile or defaults. What else a catalog entry must contain depends on the compilers that will read it. The reference container-assignment compiler, for instance, needs at least one component in each dependency chain to come with a Docker Compose description, so that it can place the chain in a container.

The integrator must also decide which framework-specific properties the compilers need in order to produce a valid configuration. In the reference implementation, LDIO's compilers need to know each component's role (Input, Adapter and so on), RDF-Connect's need to know where each processor's definition file lives, and NiFi's need to know whether a component is a processor, a controller service or a funnel. These properties become part of each catalog entry, expressed in the framework's own vocabulary (§1.1).

Two properties of the framework deserve particular attention. The first is whether it is multi-tenant, that is, whether its runtime can host several independent pipelines side by side in one container. Any implementation that places several steps of a pipeline in a shared container has to take this into account, and in the reference implementation it decides whether a component may be used more than once (§5.3.2). The second is which of the framework's components can receive or send data across container boundaries, and over which transport. The semantic model provides vocabulary to mark such boundary components in the catalog (§1.2), so that

bridging compilers can find them; in the reference implementation, marking them is enough for its bridging compiler to use them.

4.2. WRITING THE COMPILERS

The second activity starts by deciding which files the framework needs and which piece of knowledge produces each of them. Following the single-responsibility principle of §2.1, this usually leads to several small compilers. In the reference implementation, a framework typically has one compiler for its main configuration file, one for its Dockerfile where needed, and one per boundary component, which knows where that component's port and address live in its configuration. Mapping step configuration onto the framework's own configuration, whether as keys in a YAML file, properties in a flow definition or environment variables, is likewise the responsibility of the compilers.

Each compiler is then given its condition. Typically this is "a container runs this framework's orchestrator", sometimes combined with "compiler X has already run" when a compiler consumes another's output, while compilers that need the finished build wait for the finalize phase. For a multi-tenant framework whose steps share a container, the compilers must be able to emit one configuration per segment, as LDIO's compiler does with one pipeline file per segment. Framework-specific logic should remain visibly framework-specific: in its own folder, in its naming, and in its own file of inference rules, the rules that derive implied facts from a pipeline definition before it is validated, for example that every step of a plan is a pipeline step. Otherwise a rule written for one framework can accidentally apply to the next. Finally, the compilers are added to the runner configurations that should use them, typically both a generator and a validator configuration.

The compilers of the reference implementation are written in Python with *rdfine*, a lightweight package developed within the project. It gives compiler authors a compact way to read, query and transform RDF graphs, for example as tables or as nested dictionaries, and it is what makes writing compilers against RDF practical in the first place.

4.3. WRITING THE SHAPES

The third activity is to write down every assumption the new compilers make as a SHACL shape, so that validation (§3) can check it before a build is trusted. This covers the properties the compilers rely on, such as "every component of this framework declares its role", and the topological rules of the framework, such as "pipelines are strictly linear" or "at most one input per segment". Each component should receive a configuration schema, attached to its catalog entry, and, where it is known, a description of the data it expects and produces (§3.2). Where the framework publishes configuration schemas of its own, these can be reused. Which further shapes are needed depends entirely on what the compilers assume: that a component is used only once where the framework is not multi-tenant, that a pipeline is strictly linear, that component types appear in a particular order, and so on. The rule is the same in every case: whatever a compiler takes for granted should be stated as a shape, so that a violation is caught at validation time instead of failing silently at run time. Every shape carries a human-readable message stating what it checks, which is what the pipeline author sees in the validation report.

4.4. TESTING THE NEW FRAMEWORK

Unit tests are an integral part of adding a framework, not an afterthought. They run the new compilers against pipeline definitions that use the new framework, both alone and in combination with existing frameworks, and check for one of two outcomes: either the build is as expected, down to the generated files, or the validation report flags the pipeline through the shape that is meant to catch it. A pipeline that should be rejected but compiles anyway points to a missing shape, and a pipeline that should compile but is rejected points to a shape that is too strict. Together, these tests keep compilers and shapes consistent with each other as either of them changes.

PART II: THE REFERENCE IMPLEMENTATION

The reference implementation shows what the architecture can do in practice. It consists of a catalog of components from LDIO, RDF-Connect, Apache NiFi and semantic.works; a set of compilers; a set of shapes that express what these compilers expect; and two runner configurations, namely the pipeline generator (§5) and the pipeline validator (§6). Its compilers (§2.1) fall into two groups. The core compilers take framework-independent decisions, such as resolving dependencies, assigning containers and bridging between them. The framework compilers write each framework's files.

The reference implementation is one possible implementation of the architecture, not the only one. Its decisions, for example how components are assigned to containers, are decisions of these compilers rather than of the architecture, and they are marked as *design choices* where they limit what users can do.

SUPPORTED FRAMEWORKS

The architecture is not tied to any framework, but the reference implementation supports the four frameworks described below. It does not replace them; it generates their configuration, the environment they need in order to run and, at least in part, the wiring between them.

Framework	What it is	How it is configured
LDIO (Linked Data Interactions Orchestrator) [13]	A pipeline engine for Linked Data. Each pipeline has exactly one *Input*, at most one *Adapter* that converts incoming data to RDF, any number of *Transformers*, and *Outputs* at the end.	One YAML file per pipeline.
RDF-Connect [14]	A framework for streaming Linked Data through *processors* written in JavaScript or Python. A *runner* executes the processors of one language.	One pipeline file written in RDF.
Apache NiFi [15]	A widely used general-purpose data-flow tool, normally operated through a visual editor. Its building blocks are *processors*, *controller services* (shared resources such as credentials) and *funnels* (merge points).	A flow definition in JSON.
semantic.works [16]	A collection of small, separate services (*microservices*) grouped around a shared triple store. The services communicate by writing to and reacting to changes in the store.	Each service has its own configuration, through environment variables or files in its own format.

REFERENCE CASES

The reference implementation has been tested against real pipelines that were first built by hand, and §5.2 reports how well it reproduces them. The **DiSHACLed demonstrator** monitors water levels and combines three frameworks in a single pipeline. LDIO fetches sensor readings from a web API and converts them into standard observation data, RDF-Connect checks each reading against a threshold, and semantic.works stores the results and composes an alert e-mail whenever a threshold is exceeded.

In addition, two **reference architectures of the City of Bruges**, built independently of this project, were used. The *bicycle parking* case publishes parking locations as an LDES and loads them into a triple store, using LDIO and RDF-Connect. The *water quality* case fetches monthly measurement files and publishes them as an LDES, using NiFi. The regeneration of both cases is documented in project deliverable **D2.2b**, and deliverable **D2.3b** reports timing measurements for them.

5. PIPELINE GENERATOR

5.1. HOW IT WORKS

5.1.1. Inputs and outputs

The generator takes a pipeline definition and the catalog (§1.1) and produces a complete project folder. The folder contains a `docker-compose.yml` that starts every service the pipeline needs, with a start-up order derived from the dependencies between services. It also contains the native configuration files of each framework involved: LDIO pipeline files in YAML, an RDF-Connect `pipeline.ttl`, a NiFi flow definition, and the configuration of the semantic.works services. Where a framework needs them, build files are added as well, for example an RDF-Connect Dockerfile that installs only the processors the pipeline actually uses. Finally, every project contains a validation report (§6).

5.1.2. Core compilers

The core compilers take the decisions a pipeline author would otherwise take by hand. They start by resolving dependencies: components that the pipeline author did not mention but that the pipeline needs, such as an orchestrator, a runner or a triple store, are pulled in from the catalog. They then assign components to containers. Every component that comes with its own Docker Compose description becomes a separate container, and every other component is placed in the container of the component it depends on. Next, "is preceded by" statements are turned into explicit channels.

When a channel crosses from one container to another and neither side can send or receive over the network, a bridging compiler looks in the catalog for a matching pair of HTTP boundary components and inserts them. A small compiler for each boundary component then configures its port and address: the receiving side publishes its address on the channel, and the sending side reads it from there. Each step is then tagged with the segment it belongs to, so that frameworks requiring one configuration per segment can produce it. In the finalize phase, the Compose file is assembled from the Compose descriptions of all containers.

5.1.3. Framework compilers

Each framework has its own compilers. They decide which of the framework's files the build needs and attach each of them, with its contents, to the build description, which the core compilers have shaped by then. For semantic.works, for instance, there is one compiler per service, each producing that service's configuration files. The outputs per framework are listed in §5.2.

5.1.4. Example: compiling a pipeline

The steps below compile the DiSHACLed demonstrator, the largest of the reference cases. For this example, the two steps with which the demonstrator's definition declares the bridge between LDIO and RDF-Connect by hand are left out, so that the generator has to insert the bridge itself. The definition then contains 13 steps in three frameworks: three LDIO steps that poll a water-level API and turn each reading into an observation, eight RDF-Connect steps that check each observation against a threshold and turn readings outside it into error records, and two semantic.works steps that compose and deliver an alert e-mail. Figure 4 shows the definition and the resulting build. The order of the steps below is not configured anywhere; it follows from each compiler's condition (§2.1).

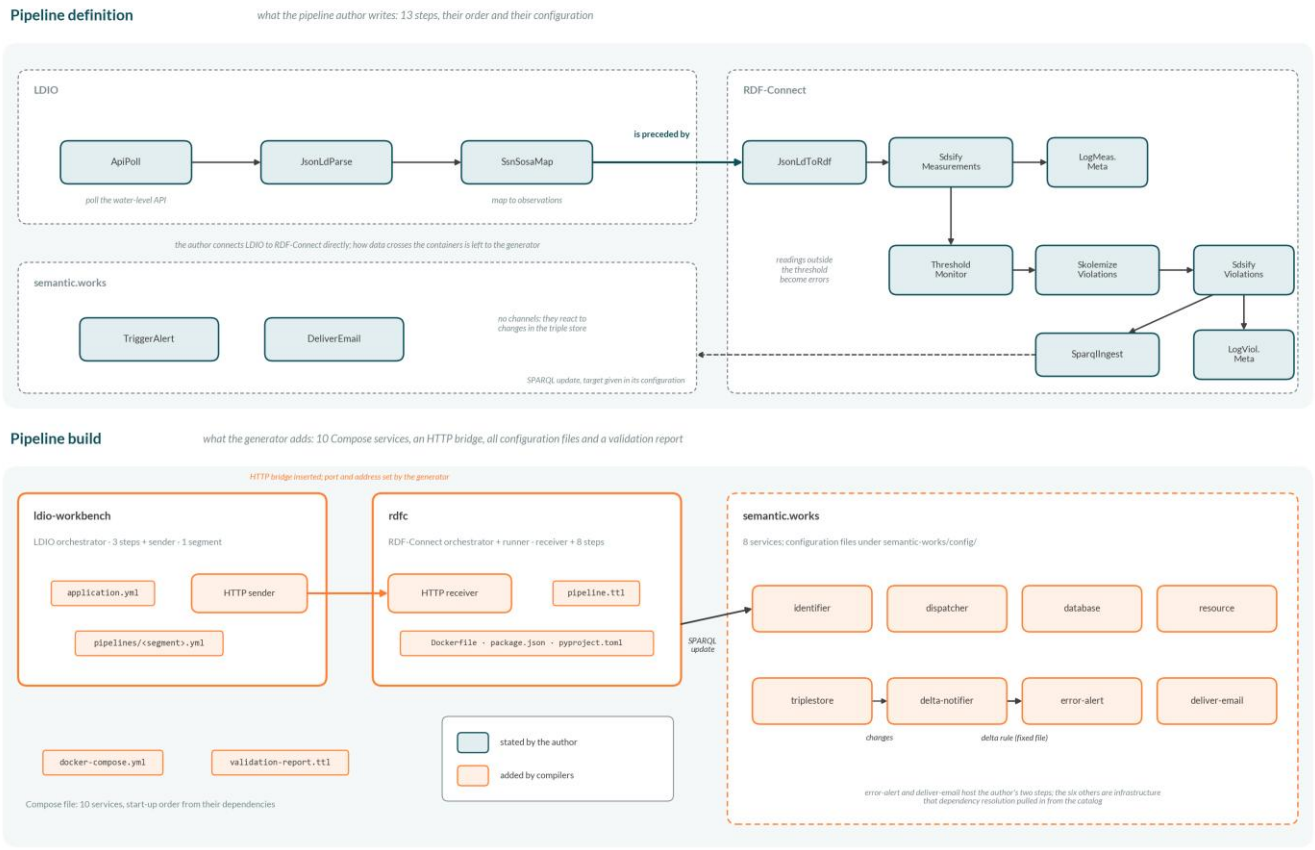

Pipeline definition
what the pipeline author writes: 13 steps, their order and their configuration
LDIO
ApiPoll
JsonLdParse
SsnSosaMap
is preceded by
poll the water-level API
map to observations
RDF-Connect
JsonLdToRdf
Sdsify Measurements
LogMeas. Meta
the author connects LDIO to RDF-Connect directly; how data crosses the containers is left to the generator
readings outside the threshold become errors
Threshold Monitor
Skolemize Violations
Sdsify Violations
semantic.works
TriggerAlert
DeliverEmail
no channels: they react to changes in the triple store
SparqlIngest
LogViol. Meta
SPARQL update, target given in its configuration
Pipeline build
what the generator adds: 10 Compose services, an HTTP bridge, all configuration files and a validation report
HTTP bridge inserted; port and address set by the generator
ldio-workbench
LDIO orchestrator - 3 steps + sender - 1 segment
application.yml
HTTP sender
pipelines/.yml
rdfc
RDF-Connect orchestrator + runner - receiver + 8 steps
HTTP receiver
pipeline.ttl
Dockerfile · package.json · pyproject.toml
SPARQL update
semantic.works
8 services; configuration files under semantic-works/config/
identifier
dispatcher
database
resource
triplestore
delta-notifier
error-alert
deliver-email
changes
delta rule (fixed file)
error-alert and deliver-email host the author's two steps; the six others are infrastructure that dependency resolution pulled in from the catalog
stated by the author
added by compilers
docker-compose.yml
validation-report.ttl
Compose file: 10 services, start-up order from their dependencies

Figure 4. The demonstrator pipeline. Top: the definition as written by the pipeline author, with its steps grouped by framework and no bridge declared between LDIO and RDF-Connect. Bottom: the build generated from it, all of which is added by compilers. Dependency resolution adds the LDIO and RDF-Connect orchestrators and six semantic.works infrastructure services; the bridging compiler inserts an HTTP sender and receiver between LDIO and RDF-Connect and configures their port and address; the framework compilers write every configuration file, the Compose file and the validation report.

1. **Seeding.** The runner receives the request to compile `demo:DishacledPipeline`, and the first compiler creates an empty build linked to this definition.
2. **Dependency resolution and container assignment.** The catalog states what each step's component needs. The LDIO steps run inside the LDIO orchestrator, the RDF-Connect steps inside a runner that in turn runs inside the RDF-Connect orchestrator, and the two semantic.works steps inside their own services, which need six further semantic.works services, such as the triple store and the delta notifier. All of these are added to the build. Every component that comes with a Compose description becomes a container, so the build gets ten: one for LDIO, one for RDF-Connect and eight for semantic.works.
3. **Wiring and bridging.** The "is preceded by" statements are turned into channels. One channel crosses containers, from `SsnSosaMap` in the LDIO container to `JsonLdToRdf` in the RDF-Connect container. Neither step can send or receive over the network, so the bridging compiler looks in the catalog for a matching pair of HTTP boundary components and inserts them: an LDIO HTTP sender after `SsnSosaMap` and an RDF-Connect HTTP receiver before `JsonLdToRdf`. The compilers for these two boundary components then configure the transport: the RDF-Connect receiver picks a port and publishes its address on the channel, and the LDIO sender reads that address and uses it.
4. **Segmenting.** Each step is tagged with its segment, the stretch of connected steps inside one container through which data flows without leaving it. Segments matter for two reasons: some frameworks need one configuration per segment, and some framework rules apply per segment, such as LDIO's rule that a pipeline has exactly one input. Here, the three LDIO steps and the inserted sender form one segment, so LDIO gets one pipeline file. In other words, segments are metadata about the build that validation can check against.

5. **Framework files.** The framework compilers decide which files the build needs and attach each of them, with its contents, to the build description. For LDIO these are the pipeline file of the segment and `application.yml`; for RDF-Connect `pipeline.ttl`, a Dockerfile and dependency lists containing only the processors in use; for semantic.works the configuration files of the services, taken from the catalog defaults.
6. **Finalize phase.** The validation report is attached, and the `docker-compose.yml` is assembled from the Compose descriptions of all ten containers.
7. **Writing files.** The files recorded in the build are written to the output folder.

The result is a complete project folder. Two parts of the demonstrator are not derived from the definition: the SPARQL update from RDF-Connect into the triple store, whose target is given in the step's configuration, and the rule that makes the delta notifier call the alert service, which ships as a fixed file (§5.3.1). One RDF-Connect processor, the JSON-LD-to-N-Quads converter, is not published and has to be copied into the project by hand before the containers are built (§5.3.1).

5.2. WHAT IT SUPPORTS

The generator supports the four frameworks listed in the table below, and pipelines may freely mix them. The reference pipelines include LDIO → RDF-Connect → semantic.works (the demonstrator), NiFi-only and NiFi/LDIO combinations, and the two-part bicycle-parking pipeline for the City of Bruges.

Framework	What the generator produces	Notes
RDF-Connect	`pipeline.ttl`, a tailored Dockerfile, and `package.json` / `pyproject.toml` listing exactly the processors used	Catalog entries are generated from the processors' own published descriptions, so adding a published processor only takes a request of three lines naming its package and version.
LDIO	One pipeline YAML per segment, plus the orchestrator's `application.yml`	Several independent LDIO pipelines can share one LDIO container. Catalog is hand-written.
Apache NiFi 2	A flow definition, a Dockerfile with the required extension bundle, and a Compose service	Can either run NiFi locally or **upload the flow to an existing remote NiFi instance**. Credentials are referenced by name and bound only at deployment. Catalog is hand-written.
semantic.works	The configuration files of the individual services (triple store, resources, dispatcher, delta notifier, authorization, error alerting) and their Compose services	Configuration is emitted from catalog defaults. Step settings are passed as environment variables. Catalog is hand-written.

Within these frameworks, the generator automates dependency resolution, container assignment, start-up order, HTTP bridges between containers (for connections whose data crosses into at most one other container) and port allocation. Everything published in a public repository is installed automatically, from container images to software packages from npm, PyPI and Maven.

The generator has been checked against the hand-built reference cases. For the DiSHACLed demonstrator, it generates the complete project from the pipeline definition: every Compose service, all 9 RDF-Connect processor types and the LDIO components the pipeline uses, and all 9 semantic.works configuration files, which are identical, down to the last character, to the hand-written ones. The generated project does not yet run without manual work, however. Besides the unpublished processor that has to be copied in by hand (§5.3.1), the generated Compose file still differs from the hand-built one in a few details: one service mounts its configuration from the wrong folder, the triple store's configuration file is generated but not mounted, and the RDF-Connect container does not publish its port. The two City of Bruges reference architectures were re-expressed and regenerated as well, and every processing component could be described and regenerated (D2.2b).

The generator adds no runtime overhead. The generated files are ordinary configuration files of the target frameworks, so at run time the pipeline behaves exactly like a hand-built one. The only cost is the generation time itself, which was measured at 17 to 31 seconds for the two Bruges cases (D2.3b).

5.3. WHAT IT DOES NOT SUPPORT

5.3.1. Not yet implemented

The limitations in this subsection could all be resolved within the current design of the compilers; §5.4 describes the intended resolution for each.

Automatic installation does not yet cover every component. Everything published in a public repository is installed automatically, but some components are not published and are shipped as source code next to the catalog instead. This applies to RDF-Connect's HTTP sender, which is needed whenever RDF-Connect has to send data to another container, and to the demonstrator's JSON-LD-to-N-Quads processor. The generator lists such a component as a dependency, but it neither copies its source code into the generated project nor writes a dependency entry that the package manager can resolve. Today, the pipeline author has to copy the source folder into the project and adjust the dependency list by hand before the containers can be built. Infrastructure without a catalog entry, such as LDES servers, reverse proxies and front-ends, is not generated and remains hand-written next to the generated project, although such components could be added to the catalog like any other.

Automatic bridge-building is limited in scope. Only HTTP bridges are inserted automatically, so a connection over another transport, such as RDF-Connect writing SPARQL updates into a semantic.works triple store, must still be described explicitly in the pipeline definition, even though the catalog already records which transport each boundary component speaks. A connection that crosses containers must have all its senders in one container and its receivers in at most one other container; merging data from several containers, or sending it to several containers at once, has to be built by hand, and the validator flags such cases. A bridge that is only half-declared, with the sending side written by the pipeline author but not the receiving side or vice versa, is rejected as ambiguous. Event-driven links in semantic.works cannot be generated because the semantic model cannot yet describe them (§1.3); in the demonstrator, the rule that routes error records to the alerting service therefore ships as a fixed configuration file. Finally, if the catalog offers more than one receiving or sending component for the same transport and framework, there is no rule yet for choosing between them.

There is no standardised way yet to configure semantic.works services. Each service is configured differently, some through environment variables and others through files in their own formats, such as Lisp, Elixir, JavaScript, JSON or INI. The compilers do not translate step configuration into these formats. They pass step settings on as environment variables and otherwise copy the default files from the catalog. If two steps set the same environment variable on one service, the last one silently wins. Because defaults are emitted unchanged (§5.3.2), several semantic.works defaults currently contain content specific to the demonstrator, including the alert e-mail text, mail-folder identifiers, a single public graph with open read/write access, and minimal dispatcher routes. Any other pipeline that uses these services receives the same content.

The catalogs also require maintenance. Only the RDF-Connect catalog is generated from the components' own published descriptions. The LDIO, NiFi and semantic.works catalogs are written by hand, because those ecosystems publish no machine-readable component descriptions, and they can therefore drift from the real components.

Although NiFi flow generation and remote deployment are implemented, they have not yet been verified in live production use.

A catalogue of edge cases kept alongside the generator's tests records a few more pipelines that are known to be handled incorrectly. A step that has neither explicit connections nor a predecessor is compiled without being connected, and nothing flags it (§6.3). When two plans described together share a component and each assigns it its own container, the result is undefined, and no check guards against it. Both gaps can be closed by adding shapes: one that flags a step without any connection, and one that rejects a component that several plans share but assign to different containers. For RDF-Connect's SPARQL ingest step, a validation rule demands a setting that the generator only adds later, so a correct pipeline receives a false violation; this is an instance of the missing distinction between user-facing and compiler-facing configuration (§3.4). Here a further shape would not help, since the shape itself is what misfires; the case needs that distinction.

5.3.2. Design choices of the reference implementation

Two limitations follow from deliberate decisions in the reference compilers. Each could be changed by writing different compilers, without any change to the architecture.

The first is that there is **one container per service component**. The container-assignment compiler creates exactly one container for each catalog component that comes with a Docker Compose description, and it places every component that depends on it in that same container. The assignment depends only on the dependencies recorded in the catalog, never on the structure of a particular pipeline. As a result, there is no way to request several separate containers of the same component, for example to isolate two pipelines from each other or to scale out a single step. It also means that a framework must either be multi-tenant, or each of its components may be used only once per pipeline. LDIO and RDF-Connect are multi-tenant, but semantic.works services are not, since each owns a single static configuration. Using the same semantic.works component twice in one pipeline is therefore flagged by the validator rather than silently producing a broken result. The rule was chosen because it is simple and also works for infrastructure components that run no pipeline step at all, such as a triple store or a gateway. A different assignment compiler could take the structure of each pipeline into account, but it would also need vocabulary for deployment choices in the semantic model (§1.3).

The second is that **catalog defaults are emitted unchanged**. Configuration files stored as defaults in the catalog are written out exactly as they are, and a pipeline can neither replace a default with its own version nor merge changes into it. Defaults are therefore only appropriate for genuinely generic content. Anything pipeline-specific has to be expressed as step configuration, which only works where the framework and its compiler support it. Changing this requires an override mechanism in the semantic model (§1.4) and support for it in the compilers that emit defaults.

5.3.3. Limits set by the target frameworks

Some limits come from outside the toolchain. The toolchain generates configuration for existing frameworks, and it cannot be expected to do more than those frameworks can: each framework's own rules carry over to every pipeline generated for it. LDIO pipelines, for example, are strictly linear, with exactly one input and at most one adapter, so branching has to happen in another framework or across segments. Strictly speaking these are not limitations of the toolchain, but they determine what a pipeline author can build with it.

5.4. FUTURE DIRECTIONS

Three directions matter most for the generator. The first is a more complete configuration of semantic.works: a schema for each service's configuration, and a standard way to pass step settings into it, would remove the remaining manual configuration and let the validator check it. Together with an override mechanism for catalog defaults (§1.4), it would also allow the content that is currently specific to the demonstrator, such as the text of the alert e-mail, to move out of the shared catalog and into the demonstrator's own pipeline definition. The second is the split between user-facing and compiler-facing configuration (§3.4), which would let the pipeline author configure a step in terms suited to them while the compilers receive exactly what they need. The third is full round-trip bridge building: bridges for every transport the catalog knows, including SPARQL updates and message brokers such as Kafka, for merging and fanning out across containers, and for event-driven links such as those of the semantic.works delta notifier, all derived from the pipeline definition instead of written by hand.

Beyond these, locally sourced components should be copied into the generated project and referenced by their local path, and catalog generation could be extended beyond RDF-Connect, for example by extracting component descriptions from LDIO's documentation site or from the READMEs of semantic.works services.

6. PIPELINE VALIDATOR

6.1. HOW IT WORKS

The validator is a runner configuration in the sense of §3.1. It runs the same core compilers as the generator, so that it sees the build as it will actually be produced, including inserted bridges, and then runs the validation-report compiler, but none of the compilers that write framework files. The same validation-report compiler also runs at the end of every generation, so every generated project contains a validation report.

The rules the validator checks come from three sources. Rules of the semantic model apply to every implementation, for example "every step uses exactly one catalog component". Rules of the reference compilers express what the core and framework compilers of §5 expect, for example "every connection between containers can be bridged" or "LDIO pipelines are strictly linear". Configuration schemas, finally, are attached to individual catalog components.

6.2. WHAT IT SUPPORTS

All constraint checks are operational. More importantly, the validator detects inconsistencies across frameworks that no single framework can see (D2.3b).

The table below summarises what is checked, where each group of rules comes from, and which problem it prevents. Appendix A gives the complete list.

What is checked	Source	Problem it prevents
The pipeline is complete and consistent. Every step uses a component that exists in the catalog, has at most one configuration, and belongs to exactly one pipeline. Data never flows in a circle.	Semantic model	A pipeline that refers to missing components, or that loops forever.
Configurations and secrets are well-formed. A step's configuration follows the schema of its component, where the catalog provides one. Secrets are referred to by the name of an environment variable.	Semantic model, component schemas	Misspelled or missing settings that would only surface when the framework loads its configuration; secret values ending up in files.
Every component can actually be deployed. Each component either has its own container description or depends on one that does. Software packages name the repository they come from.	Reference compilers	A generated project that cannot start because some part has nowhere to run or cannot be installed.
Connections between containers can be built. A suitable sender and receiver exist for every connection that crosses containers, and the connection has a supported layout.	Reference compilers	A pipeline whose parts in different containers cannot reach each other.
Each framework's own rules are respected. For example: LDIO pipelines are strictly linear and start with an Input; an RDF-Connect processor can be found by its runner; NiFi references point at valid services; semantic.works components are used only once.	Reference compilers	Configurations that the framework would reject at start-up, or that would run but behave incorrectly.

6.3. WHAT IT DOES NOT SUPPORT

Beyond the gaps of the validation principle itself (§3.4), the reference validator has limitations that are not yet implemented. Because the shape-matching library is not yet available, it collects every sender and receiver pair but reports each one as unverified. Configuration schemas and data descriptions are not yet available for every catalog component, so a step whose configuration is not covered by any schema, or a connection without a data description, cannot be fully validated. A step that has neither explicit connections nor a predecessor is not flagged either.

6.4. FUTURE DIRECTIONS

The most effective improvement is to write configuration schemas and data descriptions for every component in the catalog, since every shape added directly widens what the validator can check. Individual gaps, such as the check for unconnected steps, should be closed as well. Once the shape-matching library is available, it should be connected, although its practical use will remain limited to the few components whose input or output is fixed (§3.2).

PART III: ASSESSMENT

7. STATUS AT A GLANCE

The table below summarises Parts I and II, with one row per goal from the Motivation. For each goal it lists what works today, what could be supported but has not been implemented yet, and which constraints remain. Each constraint is marked with its origin: the architecture, the reference implementation (a design choice that other compilers could make differently), or the catalog, whose content is the responsibility of its maintainers.

Goal	Supported today	Not yet implemented	Constraints
Interoperability (§5.2, §5.3)	Components of RDF-Connect, LDIO, NiFi and semantic.works can be combined in one pipeline. When data has to pass between containers over HTTP, the sending and receiving components are added automatically.	Automatic connections between containers over anything other than HTTP. Merging data from several containers, or sending it to several containers at once. Services in semantic.works that react to changes in the triple store.	—
Replicability (§2.3, §5.2, §5.3.1)	The complete project can be regenerated from its description at any time. The hand-built demonstrator is largely reproduced; a few details still need manual correction.	Including services that are already running, such as an existing triple store.	Catalog: components that exist only as local files cannot be replicated elsewhere. Ideally, catalog maintainers publish every file a component needs and announce it in the catalog.
Automatic validation (§3, §6)	Before deployment, many errors are detected, including errors in how pipeline parts in different frameworks fit together, with the offending part named.	Checking that the data one component sends matches what the next one expects (a supporting library is still in development, and its practical use is limited, §3.2). Reporting which parts could not be checked. Separating what the pipeline author must configure from what compilers need. Tying each rule to the compiler that needs it, so that rules follow the compilers actually in use. Rules for every catalog component. Checking live data while the pipeline runs.	Architecture: only rules that have been written down can be checked, so a passing report does not guarantee a working pipeline. What a component does to the data cannot be described in SHACL, so errors in transformation logic cannot be detected before the pipeline runs, and data checks are only useful if the data descriptions match the data that actually flows.
Automatic installation (§5.2, §5.3.1)	Everything published in a public repository is installed automatically: container images, and software packages from npm, PyPI and Maven. RDF-Connect containers include only the processors the pipeline uses.	Components that are not published in a public repository; they currently have to be copied into the project by hand. Infrastructure such as LDES servers or front-ends.	—

Goal	Supported today	Not yet implemented	Constraints
Ease of use (§1.3, §5.1, §5.3)	Dependencies, containers, ports, connections between containers and every framework's configuration format are handled by the tool.	Replacing a component's default configuration files with pipeline-specific ones. Pre-configured templates that help authors configure a step. A uniform way to configure semantic.works services. A graphical front-end or a language model for writing pipeline definitions.	—
Semantics (§1)	Pipelines, components and builds are described as Linked Data, including a full record of how each build was produced.	Relating descriptions written against different versions of the model. Describing conditional routing ("on failure, send here"), other transports, reactions to events, and deployment choices.	Architecture: SHACL cannot describe how a component transforms data (§3.2).
Modularity (§2, §4)	Compilers and frameworks are added without changing existing ones. NiFi was added this way during the project. RDF-Connect catalog entries are generated from the processors' own published descriptions.	Generating the catalog automatically for LDIO, NiFi and semantic.works. Loading the catalog on demand, so that the runner scales to large catalogs.	Architecture: the order of compilers emerges from their conditions. Reference implementation: every service component runs in exactly one container, so pipelines cannot be isolated from each other and components cannot be scaled out, and components of frameworks that are not multi-tenant (semantic.works) can be used only once per pipeline.

8. CONCLUSION

The toolchain specification has reached a working state on both layers. The architecture, consisting of a semantic model, a compilation runner and a validation principle, is in place and contains no knowledge of any particular framework. The reference implementation built on it generates multi-container projects from a single RDF description across four frameworks. It regenerates the hand-built DiSHACLed demonstrator, apart from a few details that still need manual correction, and has regenerated the processing components of two independently built reference architectures of the City of Bruges. This section measures what was achieved against the goals set out in the Motivation, and states plainly where the result falls short.

Interoperability has been achieved for the common case and only partly beyond it. Mixing frameworks in one pipeline works, and so does bridging containers automatically over HTTP. This was the central aim, and it has been demonstrated end to end. However, only HTTP bridges with simple topologies are automatic. Even in the demonstrator, the hop from RDF-Connect into the semantic.works triple store is written out by hand, and the link that routes error records to the alerting service is a fixed configuration file rather than something derived from the pipeline description. These are gaps of the reference implementation and of the model's vocabulary, not of the architecture.

Replicability has largely been achieved, with caveats. A project can be regenerated from its description at any time, and the demonstrator's semantic.works configuration files are reproduced down to the last character, although its generated Compose file still needs a few manual corrections (§5.2). Some of that fidelity, however, comes from catalog defaults that contain demonstrator-specific content, which another pipeline using the same

services would inherit; this is replication of a different kind than intended. Pipelines that include locally sourced components still need hand edits after generation, infrastructure outside the catalog stays hand-written, and NiFi deployment has not yet been verified in production. "Regenerate with one command" therefore holds for many pipelines, but not yet for all.

Automatic validation has been achieved for structure but not yet for data. The validator catches a broad range of structural, cross-container and framework-specific errors before deployment, and names the offending part, including errors that no single framework can detect. It does so in about three seconds for the reference pipelines, which D2.3b found to shorten the feedback loop by a factor of roughly 2.5 for a single LDIO container and up to 6 for NiFi, compared with starting the containers to discover a configuration error. The most ambitious part of the goal, checking that the data one component produces matches what the next one expects, is prepared end to end but does not yet produce verdicts. More fundamentally, the project found this part of the goal to be closer to a theoretical exercise than to a practical tool: most components cannot state in advance what data they expect or produce, because their behaviour is defined by their configuration, and SHACL, being static, cannot express that dependency (§3.2). Configuration schemas and data descriptions are also missing for part of the catalog, and the report does not yet say what went unchecked, so a passing report is currently weaker evidence than it appears. Beyond these fixable gaps, one limit is inherent: the validator can only check what has been written down. Even so, stating constraints explicitly and checking them automatically across frameworks goes well beyond what the frameworks offer on their own.

Automatic installation has been achieved for published components. Images, packages and extensions are pulled from public registries, and RDF-Connect images contain only what the pipeline uses. The notable exception is components that are not published, which have to be copied into the project by hand. This exception is more than a corner case, because RDF-Connect's own HTTP sender is one of these components, so every pipeline in which RDF-Connect sends data to another container is currently affected.

Ease of use is substantial, but less than intended. The tool takes a great deal of work off the pipeline author, including dependency resolution, container layout, ports, bridges and every framework's configuration format. Still, the pipeline author writes the pipeline description in RDF by hand, which requires familiarity with the semantic model, and step configuration uses each framework's own properties, so that configuring an LDIO poller, for example, requires knowing LDIO's parameter names. Catalog defaults cannot be overridden, and there are no configuration templates yet. As D2.3b argues, an experienced user of a single framework may still be faster building a pipeline by hand. The advantage of the toolchain lies in working across frameworks from one description, not in raw speed for single-framework work. At the same time, the pipeline definition is an interface for further automation. A graphical front-end could translate a visual design into a definition, or a language model could write one from a request in plain language; the generator then takes care of the rest, and the built-in validation checks the result (§1.4). The hard part, turning a design into a working multi-framework deployment, is thereby already automated; what remains is making the description itself easier to write.

For **Semantics**, the foundation has been laid, but the benefits have not yet been exercised. Pipelines, components and builds are described in RDF with established vocabularies, and every build records its full provenance. The benefits the Motivation promised beyond execution, namely discoverability through shared catalogs and analysis across pipelines, have been demonstrated only in principle. The catalog is local to the repository, and alignment with the discovery specification is still outstanding. The model enables these uses, but they have not yet been put into practice.

Modularity has been achieved. The architecture has passed its most concrete test: Apache NiFi was added as a fourth framework during the project without changes to the compilation runner or to the other frameworks. Components can be added through the catalog alone, and a published RDF-Connect processor needs only a request of three lines naming its package and version, from which its catalog entry is generated. The cost of this modularity lies in what the compilers capture: the knowledge of how a pipeline definition becomes a working implementation, which today is applied by people, use case by use case. Adding a framework means writing this knowledge down once as compilers and shapes, which is developer work. The *rdfine* package, developed within the project, makes that work feasible in the first place (§4.2). The catalogs outside RDF-Connect are maintained by hand and can drift from the real components. Scalability, finally, is limited by the current runner, which loads the whole catalog and asks every compiler in every round, although both can be improved without changing the architecture (§2.3).

Separating the architecture from the reference implementation shows that most limitations lie in the latter. The majority of the remaining gaps are simply not yet implemented: non-HTTP bridges, locally sourced components, overridable defaults, data-compatibility verdicts, coverage reporting, catalog generation for further frameworks, and version mapping. Two limitations are design choices of the reference implementation, namely one container per service component, which limits isolation and scaling and rules out reusing components of single-tenant frameworks, and the emission of defaults without change. Both could be changed by writing different compilers. Only a small set of constraints is inherent to the architecture: the emergent ordering of compilers, the dependence of a correct build on how completely the compilers' assumptions are written down as shapes, and the limits of SHACL. The target frameworks set further limits, but these lie outside the toolchain.

The second of these architectural properties is also the approach's main strength. Because the knowledge about each framework is split into small compilers with explicit, checkable expectations, every shape added widens the range of errors the toolchain catches before a pipeline ever runs.

REFERENCES

[1] RDF 1.1 Concepts and Abstract Syntax. W3C Recommendation, 2014. https://www.w3.org/TR/rdf11-concepts/

[2] RDF 1.1 Turtle: Terse RDF Triple Language. W3C Recommendation, 2014. https://www.w3.org/TR/turtle/

[3] SPARQL 1.1 Overview (including SPARQL 1.1 Update). W3C Recommendation, 2013. https://www.w3.org/TR/sparql11-overview/

[4] Linked Data Event Streams specification. SEMIC. https://w3id.org/ldes/specification

[5] Shapes Constraint Language (SHACL). W3C Recommendation, 2017. https://www.w3.org/TR/shacl/

[6] Docker Compose documentation. Docker Inc. https://docs.docker.com/compose/

[7] Toolchain specification: semantic model and pipeline generator. DiSHACLed project, 2026. https://github.com/thcarsten/toolchain-specification

[8] D. Garijo and Y. Gil. The P-Plan Ontology, 2013. https://vocab.linkeddata.es/p-plan/

[9] Data Catalog Vocabulary (DCAT), Version 3. W3C Recommendation, 2024. https://www.w3.org/TR/vocab-dcat-3/

[10] PROV-O: The PROV Ontology. W3C Recommendation, 2013. https://www.w3.org/TR/prov-o/

[11] The System Package Data Exchange (SPDX) specification. The Linux Foundation. https://spdx.dev/use/specifications/

[12] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. Introduction to Algorithms, 4th edition (chapter on elementary graph algorithms). MIT Press, 2022.

[13] VSDS Linked Data Interactions (LDIO) documentation. Digitaal Vlaanderen. https://informatievlaanderen.github.io/VSDS-Linked-Data-Interactions/

[14] RDF-Connect. https://rdf-connect.github.io

[15] Apache NiFi. The Apache Software Foundation. https://nifi.apache.org/

[16] semantic.works. https://semantic.works/

APPENDIX A. COMPLETE LIST OF CHECKS

This appendix lists the checks the reference validator performs, as of the date of this report. Terms are explained in the Background section. The checks fall into three kinds.

The first kind consists of the general rules in the table below, which are evaluated for every pipeline. The group marked "Semantic model" contains rules of the model itself, which apply to every implementation. The remaining groups express assumptions of the reference compilers.

Area	Check
Semantic model	Every pipeline has at least one step.
Semantic model	Every step belongs to exactly one pipeline and uses exactly one catalog component, and that component is listed in a catalog.
Semantic model	A step has at most one configuration, and the connections it reads from or writes to are real channels.
Semantic model	Data never flows in a circle: no step reads, directly or indirectly, its own output.
Semantic model	Every catalog entry is a proper pipeline component. Its configurations are real configurations, and its dependencies point at other components or at software packages.
Semantic model	A component has at most one default configuration of each kind.
Semantic model	Every configuration is given either inline or as a text body, never both and never neither, and a text body declares its format.
Semantic model	Secret references use valid environment-variable names, so no secret value has to be written into the definition.
Semantic model	The generated build links to exactly one pipeline definition. Every container runs at least one component, and its parts are real containers, steps and files.
Deployment	Every component either comes with a Docker Compose description or depends, directly or indirectly, on a component that does.
Deployment	Software packages declare a name and the package manager that supplies them.
Connections between containers	Every connection that crosses a container boundary can be bridged, because the catalog offers a suitable sending and receiving component.
Connections between containers	The connection has a supported layout: all its senders sit in one container, and its receivers in at most one other container.
Connections between containers	A boundary component declares at most one transport type.
RDF-Connect	Every processor declares where its definition can be found, so that the runner can load it at start-up.
RDF-Connect	Every runner belongs to the RDF-Connect orchestrator, and the orchestrator comes with both a Compose description and a Dockerfile.
RDF-Connect	Software packages in the RDF-Connect container are supplied by a supported package manager (npm or pip).
RDF-Connect	Mandatory input and output connections of a processor are actually wired.
RDF-Connect	Where a step's connections appear both in its RDF-Connect configuration and in the framework-neutral description, the two agree.
LDIO	Every LDIO component declares exactly one role (Input, Adapter, Transformer or Output) and has a label, which becomes the pipeline's name.
LDIO	LDIO pipelines are strictly linear: each step has at most one incoming and one outgoing connection.
LDIO	Within an LDIO segment, an Input comes first and an Output comes last.
LDIO	A segment has at most one Input and at most one Adapter.
Apache NiFi	The deployment mode is either local or remote, and a remote deployment has exactly one deployment configuration.
Apache NiFi	Each NiFi catalog component is exactly one kind of thing (a processor, a controller service or a funnel) and carries the complete metadata for that kind.
Apache NiFi	References to controller services point at valid controller-service steps in the same pipeline.

Area	Check
Apache NiFi	Each route references exactly one connection, which its step actually writes to, and selects at least one relationship that its component declares.
Apache NiFi	A NiFi connection has at most one writer and at most one reader, since merging and fan-out are not yet supported, and no step reads from a connection it writes to.
Apache NiFi	Each step's scheduled state is valid for its kind: running or disabled for processors, enabled or disabled for controller services, and none for funnels.
semantic.works	Every semantic.works component comes with its own Compose description.
semantic.works	Step configuration values are plain values, since they become environment variables.
semantic.works	No semantic.works component is used more than once in a pipeline (see the design choice in §5.3.2).

The second kind consists of the configuration schemas of individual components. They are stored in the catalog next to the component they describe and are checked for every step that uses that component. At present the catalog contains configuration schemas for about twenty LDIO components, nine RDF-Connect components and nine NiFi components. The semantic.works components have none yet, simply because no one has written them for the catalog so far. A schema typically states which settings a component accepts, which of them are required, and what type of value each takes. Some go further. The RDF-Connect schemas, for example, check that data-path expressions are ones the RDF-Connect runtime can actually interpret.

The third kind consists of data descriptions: shapes that describe the data a component expects or produces (§3.2). A few of them are stored in the catalog, and others are attached to steps in the pipeline definitions of the demonstrator and the bicycle-parking case. They are collected and paired for every connection, but they are not yet judged.

In addition to these checks, several compilers refuse to continue when they detect a conflict that the validator does not cover. Examples are a bridge declared on one side only, two compilers producing the same file, two components declaring the same Compose service or network name, and two components requiring conflicting versions of the same software package. These conflicts stop the build with an error message instead of appearing in the validation report. The intended principle is simpler: everything that makes a pipeline unsupported should be expressed as a shape, so that it appears in the validation report, and errors should be reserved for failures of the compilation process itself, such as a value of the wrong type. The conflicts listed above do not follow this principle yet. Moving them into shapes would give the pipeline author one complete report instead of a build that stops at the first problem.